Sequential Processing Programs

- Easier to develop but slower
- Speedup on uni-processor is achieved without involvement of programs.

Parallel Processing Programs

- Hard to develop but fast
- Programmers must become parallel programers to recieve good speed-up.

Parallel programs are hard to write because:

1. Scheduling:
   - Jobs must be broken into sub-tasks so that each processor has to do same amount of processing at the same time
2. Load balancing:
   - Must balance the loads evenly to get the desired speedup
3. Time for synchorization:
   - Job output may not be produced untill all sub-tasks are completed
4. Communication Overhead
   - Synchronization requires communication, the overhead of which must be minimal

Embarassingly Parallel Computation

- One that can be obviously divided into completely independent parts that can be executed simultaneously.
- There is no interaction b/w processes

- Results must be combined in some way
- Have the potential to achieve maximum speed up on parallel platforms.
- In nearly embrassingly parallel computation, results must be distributed & combined in some way

Amdahl's Law

$$S = \frac{1}{f + \frac{1-f}{p}}$$

Where, p

T_1: execution time on 1 processor

f: fraction of program that is sequential T_p: $\frac{fT_1 + (1-f)T_1}{p}$

p: no. of processors.

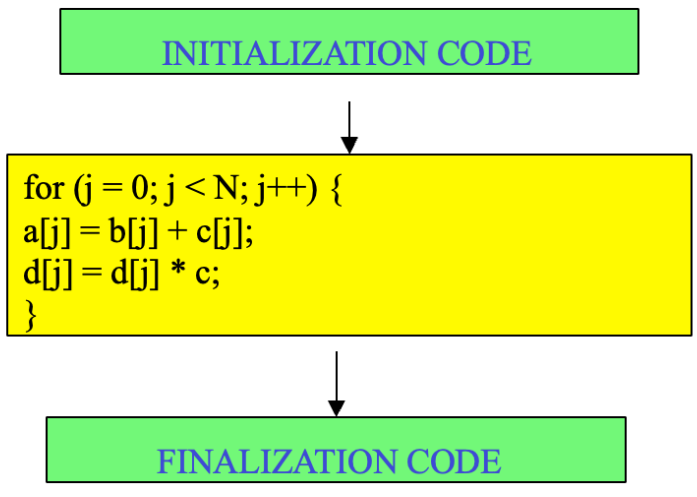
As $p \to \infty$, then $\frac{1-f}{p} \to 0$ making

$$S = \frac{1}{f}$$

Hence,

Speedup bound is determined by the degree of sequential execution time.

A program made up of 10% serial initialization and finalization code. The remainder is a fully parallelizable loop of N iterations.



$$T = T_{INIT} + T_{LOOP} + T_{FINAL} = T_{SERIAL} + T_{LOOP}$$

Diminishing returns:

- Adding more processors lead to diminishing returns
- 16 processors ≠ 16x speedup
- Serial sections of code takes a larger percentage of execution time.

We still use parallelism because

1. Throughput Computing:
   Run large no. of independent computations on different cases.
2. Scaling the problem size
   ↳ Solve larger problem sizes in given amount of time.

Serial fraction f drops as problem size is increased.

3. Sophisticated parallel algorithms/compiler techniques are able to parallelize what used to serial in the past.